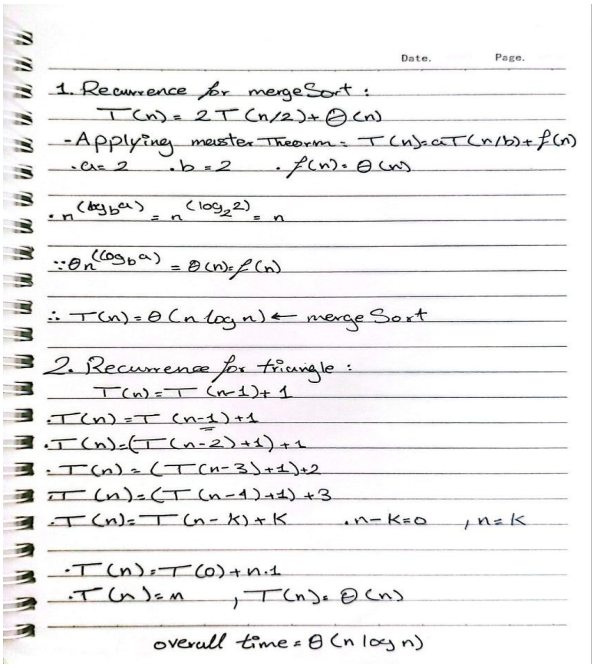
Recursive Algorithm Analysis

1. Algorithm Description:

The algorithm first creates a copy of the input array and sorts it in ascending order using merge sort. It then calls a recursive function `triangle(B, 0)` that scans the sorted array. At each step, it checks if the current index `i` allows for a valid triplet (`i < size - 2`). If so, it evaluates the triangle condition `B[i] + B[i+1] > B[i+2]` . If true, it returns `1` . Otherwise, it recursively advances to `i + 1` . If the base case `i >= size - 2`) is reached without finding a valid triplet, it returns `0` .

2. Time Complexity Analysis:



Conclusion:

- Copying the array takes: `O(n)`
- Sorting the copied arrayusing merge sort takes: `O(n log n)`
- The recursive traversal makes at most `O(n)` calls, with each call performing `O(1)` work
- Total time before returning: `O(n) + O(n log n) + O(n)`

4. Final Complexity: `O(n log n)` Because merge sorting dominates the overall time complexity. The linear recursive scan and array copy do not change the asymptotic bound.

5. Correctness Explanation:

After sorting:

`B[i] ≤ B[i+1] ≤ B[i+2]`

So we only need to check:

`B[i] + B[i+1] > B[i+2]`

The recursion systematically checks every consecutive triplet from left to right. If the condition is true for any index `i` , a triangular triplet exists and `1` is returned immediately. If the base case is reached, `0` is returned, correctly indicating no valid triplet exists. The initial `n < 3` guard clause safely handles inputs that cannot possibly form a triangle.

6. Conclusion:

The recursive approach preserves the optimal `O(n log n)` time complexity of the iterative version but trades memory efficiency for code elegance.
it may risk stack overflow for very large inputs. It is best suited for moderate array sizes or environments where recursion depth limits are not a constraint.